

DriftGuard: Reliable Network Anomaly Detection Under Distribution Shift

Te
net

Dataset: synthetic
Experiment: 20260925T233012Z_temporal_synthetic_a55cdd
Git commit: 7da118fc7ae23960339ba5683be6d042246a8dd7
Seed: 42
Generated: 2026-09-25 23:40 UTC

All numbers in this report are read from the recorded experiment directory
(20260925T233012Z_temporal_synthetic_a55cdd). No metric is written by hand.

This report evaluates whether a machine-learning network anomaly detector keeps working when the traffic it was trained on is replaced by later traffic. A detector is fitted on the first 20,000 flows of synthetic, evaluated on a held-out backtest inside the historical distribution, and then scored unchanged on 8,000 later flows it never saw. Operating thresholds come from the validation split alone, under a fixed false-positive budget. Across 2 models, 2 of 2 lost F1 on the forward test. The strongest backtest model, logistic regression, reached F1 0.559 on the backtest and 0.549 on later traffic. Drift analysis raised 1182 alerts across 2176 window-feature comparisons. Adaptation strategies were evaluated against the same false-positive budget, and the results are reported per strategy rather than reduced to a ranking.

Primary: does a machine-learning network anomaly detector trained on historical traffic continue to detect attacks reliably when evaluated on later, previously unseen traffic?

Secondary: which methods for detecting and adapting to distribution shift recover detection performance most effectively while respecting a fixed false-positive budget?

The earlier version of this project asked a different question, whether traffic metadata alone carries enough signal to separate normal traffic from attacks. That work is preserved as the baseline layer: it established the metadata-only detector this project then takes forward in time.

Why I chose this question

I started with the simpler question of whether network traffic metadata contains enough information to distinguish normal traffic from attacks. Existing research then led me to a harder problem: network traffic changes over time, and machine-learning intrusion detectors may face concept drift and feature drift. The UGR.16 research and dataset were especially useful for this direction because they were designed to study network traffic behaviour over time.

The literature motivated the problem. The exact research question and the experiment design are my own, and the project does not claim to invent a new machine-learning algorithm: the models are deliberately ordinary baselines, and the contribution is a controlled measurement of how far they degrade and whether adaptation recovers any of it.

Shyaa et al. (2024) survey concept-drift and feature-dynamics-aware machine and deep learning for intrusion detection, and give the premise of this project: a detector trained on one traffic distribution meets a different one in operation, and the resulting drift is a first-class problem rather than a deployment detail.

Macia-Fernandez et al. (2018) introduce UGR'16, whose stated purpose is evaluating intrusion detection systems that account for long-term traffic evolution and periodicity, with data gathered from a real ISP over several months. That is the dataset design this project's temporal evaluation follows.

Xu et al. (2024) address drift detection, interpretation and adaptation together, and Yang et al. (2025) propose self-supervised adaptation to concept drift for network intrusion detection. Both establish that adaptation under drift is an active problem with known partial solutions, which is why this project evaluates several strategies rather than assuming one.

Threat model

- The defender observes flow metadata only: start time, duration, packet and byte counts, protocol and TCP flags. Payload contents never inspected.
- The attacker generates malicious traffic and may encrypt, obfuscate or imitate legitimate flows.
- The environment changes independently of any attack: new software, new services, different traffic mixes. This means the defender must adapt to a changing environment.
- The software is not assumed to contain attacks the defender has not seen, and detection must not depend on prior knowledge of the specific attack.

A temporal experiment is only meaningful if nothing from the future can reach the model. Five guards enforce that here, and each is covered by a test that fails if the guard is removed.

- Periods are contiguous in time and ordered. An identical timestamp is never split across a boundary, because the split timestamp would be rows in the same period only. Fitting on all data and then splitting would leak the forward period into the backtest period.
- The operating times and sequence of validation steps from the backtest period and forward period becomes a random one.
- The backtest is used for exactly one thing: measuring how much the model decays before the forward test is touched at adaptation.
- Adaptation is given one score once.

adaptation re-fits on a cadence, and at each step the fit may only use rows earlier than the step it is about to score.

Both datasets are public and are used unmodified. What each is allowed to support is not the same, and the difference matters for how far the results travel.

What can support

UNSW-NB15 is a public dataset of network traffic captured over 23 weeks in 2015. It is a subset of the full release, which is stated below and is not the full release.

The UGR'16 subset, exactly

Full release: ~215 GB across 23 weekly archives. Subset used here: ~40 GB across 6 weekly archives. No claim is made about the weeks that were not downloaded.

A contiguous run of test weeks is needed so the forward test is later traffic. August weeks 1-3 give three consecutive weeks with both normal and attack flows, with july week5 as the earliest point and the two calibration weeks providing background-only traffic. This covers roughly one month, not the dataset's full six-month span.

- Calibration weeks (background only, never used for training): march_week3_csv.tar.gz, may_week1_csv.tar.gz.
- Weeks carrying the temporal experiment: july_week5_csv.tar.gz, august_week1_csv.tar.gz, august_week2_csv.tar.gz, and august_week3_csv.tar.gz.
- This subset contains the full UGR'16 release. No claim is made about the weeks that were not downloaded.
- The calibration weeks contain background traffic only, so no model is trained on them; they exist to characterise normal behaviour.
- UGR'16 attacks are produced by replaying malware and tools against a real ISP link, so attack realism is bounded by that link.
- The netflow export has no forward/backward direction split, which removes a set of features UNSW-NB15 provides.

The adversary here is not the attacker in the dataset. It is the passage of time, and the question is whether the measurement apparatus keeps working when the traffic it was calibrated on is replaced.

- Trusted: the historical capture is assumed to be a faithful record of what the network carried, and the labels in the dataset have assumed the same after this. The data will be either treated as hostile in the sense that nothing about it -
- Demystified: the adversary knows the data, but its labels are assumed to be uncorrupted and its content is not -
- Notified: the adversary knows the data, but its labels are assumed to be uncorrupted and its content is not -
- The privacy claim in the narrow data. None of these is as good as a formal privacy guarantee against a traffic analyst.

This report was generated from a run on synthetic. The dataset was loaded through an explicit adapter that maps the source's own fields onto a shared flow schema; fields the source does not export are left absent rather than approximated.

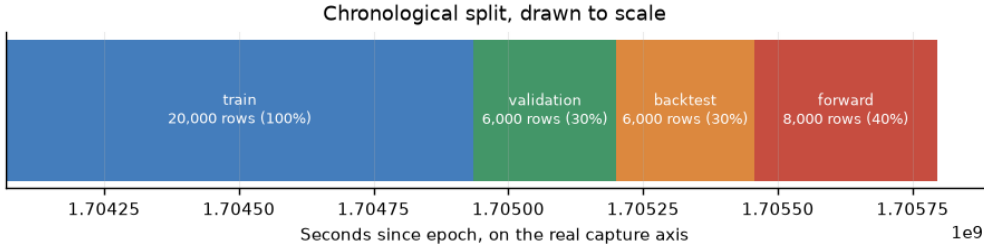
Property
Synthetic
40100rows
16model features
2024-10-01 00:00:49.394490466
2024-10-20 23:58:03.974469100
Severaldays

Dataset limitations

- The capture spans a short window. A temporal split over a few capture sessions measures relative movement between
- earlier and later captures between periods. A model that looks only for the features that may partly reflect an easier class
- Synthetic attack generation, while generated from sessions of very poor quality, captures attack variants of some attack classes
- and does not measure the quality of the results here describe this dataset under this split, and are not a claim about modern networks generally.

The feature set follows a written policy: only what a netflow or IPFIX collector actually emits, and never payload content, identity fields, or the label. The target column and the time index are excluded at selection time, and the preprocessor refuses to fit on them.

- Allowed: flow duration, packet and byte counts, derived rates, protocol, TCP flags.
- Excluded: IP addresses and ports, absolute timestamps as model inputs, the attack category, and the binary label.
- Excluded: any application-payload-derived field. A field that only exists because a tool parsed HTTP or FTP content is not available on en



The four periods on the real capture axis, drawn to scale. Every conclusion here is bounded by how short that axis is.

Periods

Period	range	Attack rate
2024-01-01	00:00:49.390000466 -> 2024-01-11 01:14:23.829091024	0.0000
2024-01-11	01:14:25.890000640 -> 2024-01-14 02:32:21.651536810	0.0000
2024-01-14	02:32:33.700000426 -> 2024-01-17 01:49:46.736657080	0.0000
2024-01-17	01:50:10.380000822 -> 2024-01-20 23:58:03.974469100	0.0000

Periods are cut by row count with boundaries snapped to real timestamps, so a single instant is never divided across two periods. The forward test is the last slice of the time axis and is scored exactly once.

Backtest methodology

The backtest is a held-out slice of the same historical distribution the model was trained on. It is the ordinary evaluation; it tells you what the detector does when nothing has changed, and it is the reference the forward result is measured against. Its score is never presented as the model's real-world performance.

Forward-test methodology

The forward test is the later slice of the time axis. The model, the scaler, the category vocabulary and the decision threshold are all fixed before it is touched, and no hyperparameter, threshold or preprocessing statistic is derived from it. Several tests enforce this, including one that fails if a training row carries a timestamp past the training cutoff and one that fails if an adaptation step receives a row from the period it is scored on.

Operating threshold and the false-positive budget

Rather than a fixed 0.5 cut, the operating threshold is chosen on the validation split to meet a false-positive budget of 5%. The threshold is the 0.95 quantile of the validation negative scores, so it depends only on how the model scores known-benign traffic, and it is then applied unchanged to the backtest and the forward test. No threshold is ever selected using forward-test labels.

Model	Validation AUC	Validation F1
majority	0.0000	0.0000
logistic regression	0.6200	0.5294

Four estimators spanning the usual complexity range, all with a fixed seed. They are ordinary baselines: the question is how they fail, not whether one can be tuned to win.

- Majority baseline: always predicts the training majority class, so a detector that cannot beat it has learned nothing.
- Logistic regression with balanced class weights: a linear reference.
- Random forest with balanced class weights: captures non-linear structure.
- Gradient boosting: sequential trees, sensitive to distribution change in a different way.

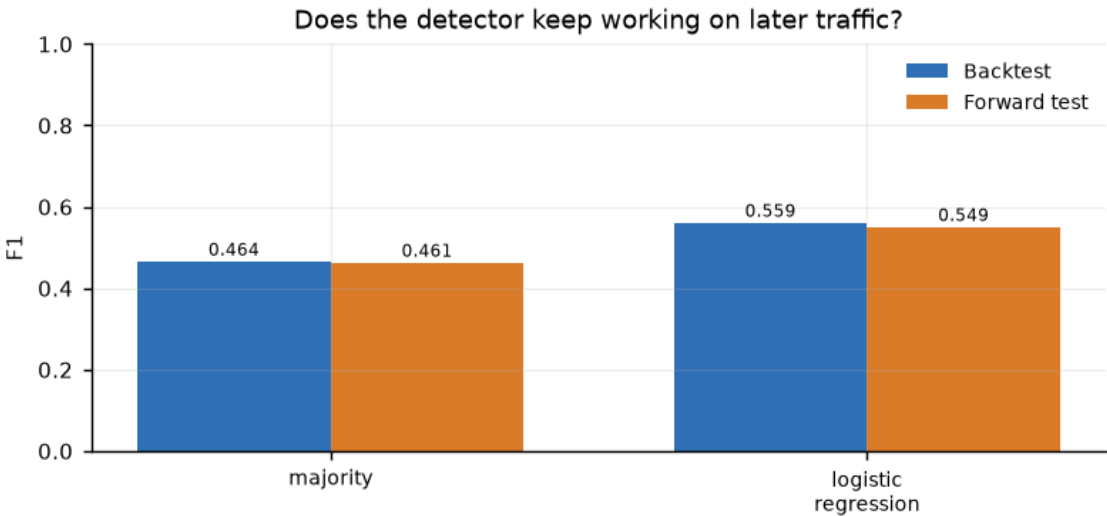
majority: trained in 0.0s, threshold 1.0000.

logistic_regression: trained in 0.0s, threshold 0.8626.

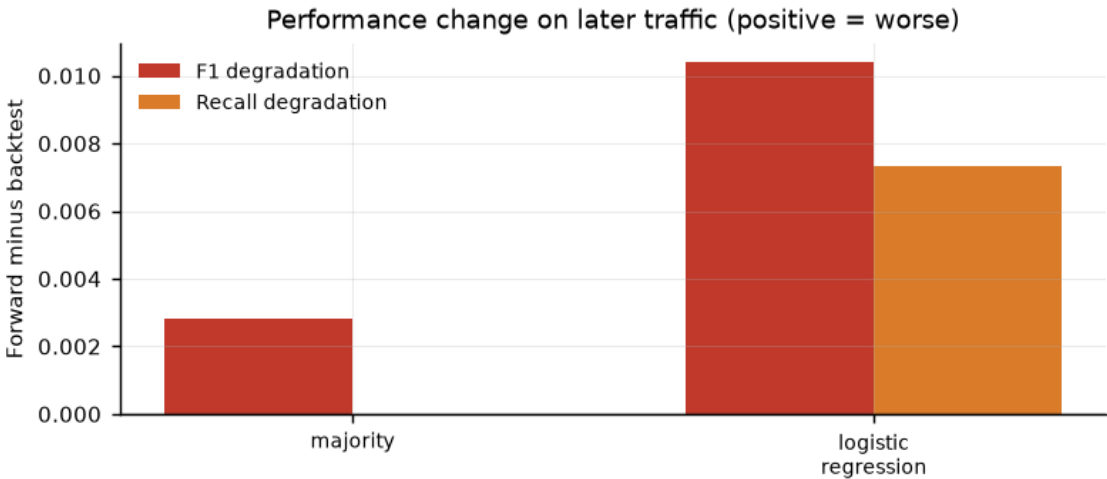
Backtest versus forward test

The two evaluations are reported separately and never merged. A positive degradation means the forward score was lower than the backtest score.

0.0000
0.0000
0.0000_regression

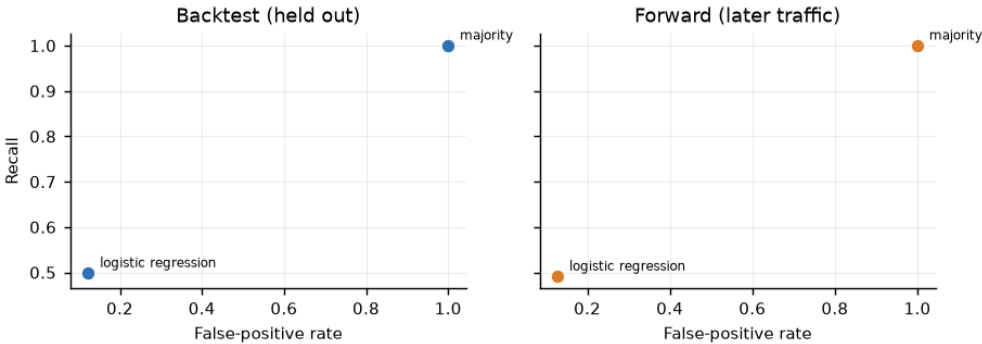


Backtest and forward-test F1 per model.



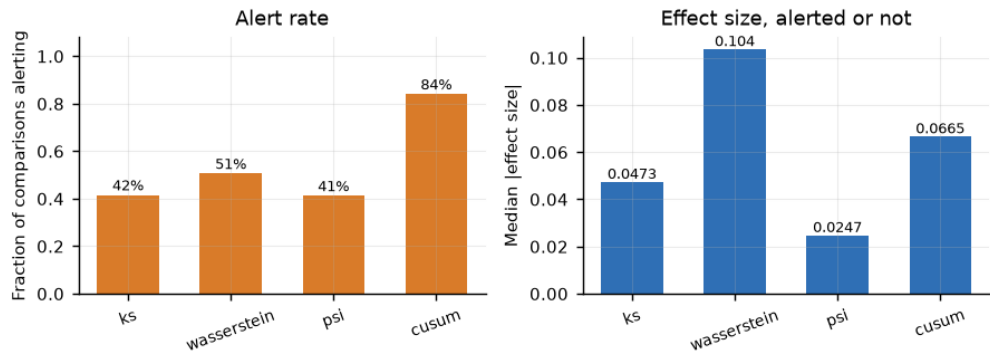
Change in F1 and recall on later traffic.

Operating points per model, at the observed degradation on validation.



Each model's operating point: recall against the false-positive rate it actually achieved, backtest versus forward.

Four detectors compared on the same windows



The four detectors on the same windows. The disagreement between them is the result; agreement would have been the surprising outcome.

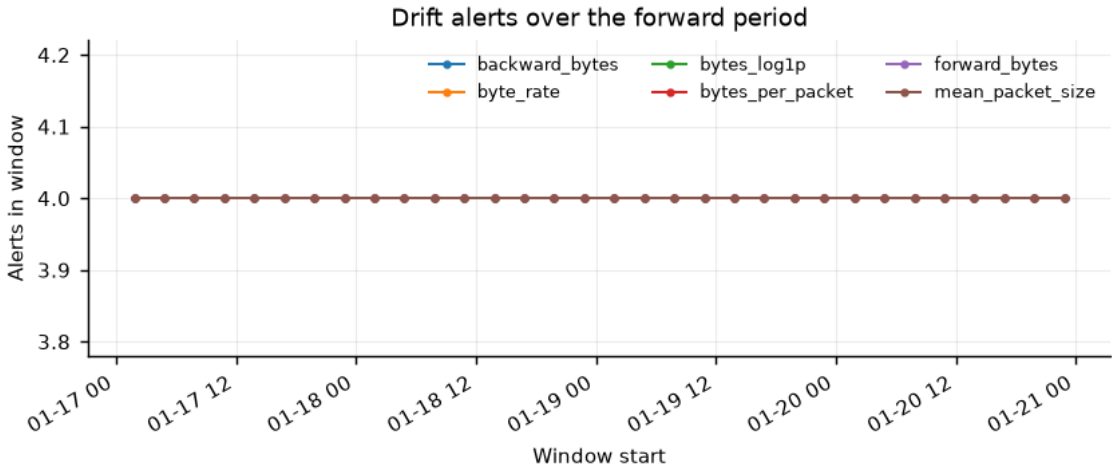
Drift is measured with a sliding window over the forward period, each window compared against the whole training period. Three two-sample methods are used - Kolmogorov-Smirnov, scaled Wasserstein distance and the population stability index - plus a sequential CUSUM detector. Window size, stride, minimum sample count and every threshold are configuration values, not constants in the code.

A p-value alone is not treated as an operational signal: with many windows and large samples, a tiny p-value fires constantly. Every method therefore also reports an effect size, and an alert requires either a test statistic above its critical value or an effect size above its configured threshold.

Method	Alerts	Comparisons
cusum	457	544
ks	226	544
psi	224	544
wasserstein	275	544

Strongest drift by feature

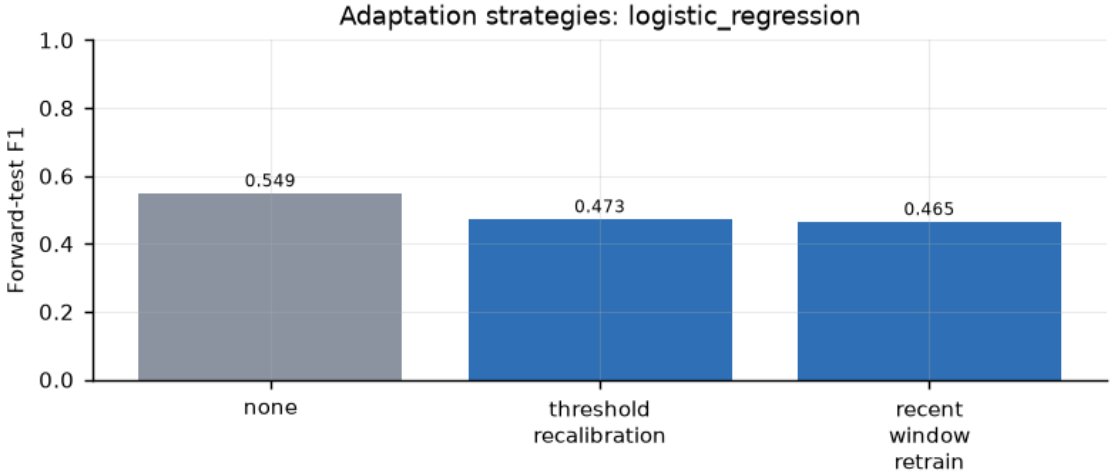
Feature	Alerts
backward_bytes	66
backward_rate	66
bytes_log1p	66
bytes_per_packet	66
forward_bytes	66
forward_packet_size	66
forward_bytes	66
forward_packets	66
duration	66
packet_rate	66
backward_packets	66
forward_packets	66
bytes_ratio	66
bytes_log1p	66



Drift alerts across the forward period.

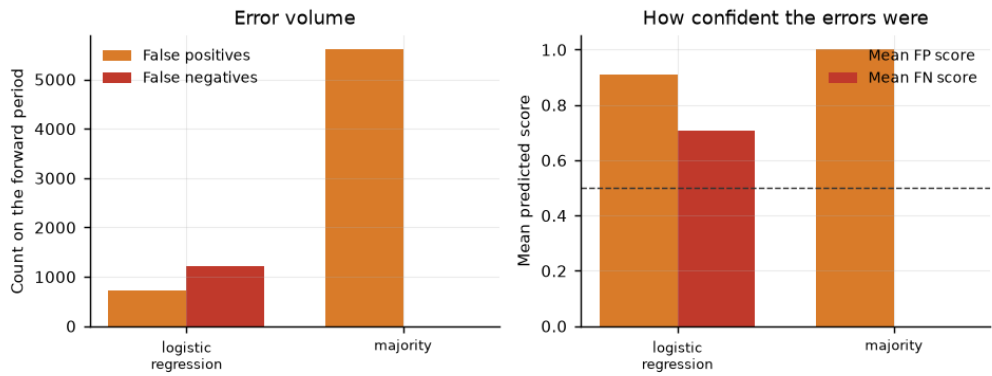
Five strategies were evaluated against the same false-positive budget: no adaptation, threshold recalibration, recent-window retraining, rolling-window retraining, and retraining on historical plus recent data. Each is permitted only rows timestamped at or before the moment it runs; a strategy that reaches into the period it is scored on raises an error and is reported as failed rather than quietly scored.

0.000000
0.000000
0.000000
0.000000
0.000000
0.000000
0.000000
0.000000



Forward-test F1 by adaptation strategy.

Recovery is expressed as a percentage of the performance lost between backtest and forward test, so a negative value means the strategy made the forward result worse than doing nothing. No strategy is declared the overall winner; the table reports what happened in this experiment.



Error volume and mean error score per model on the forward period.

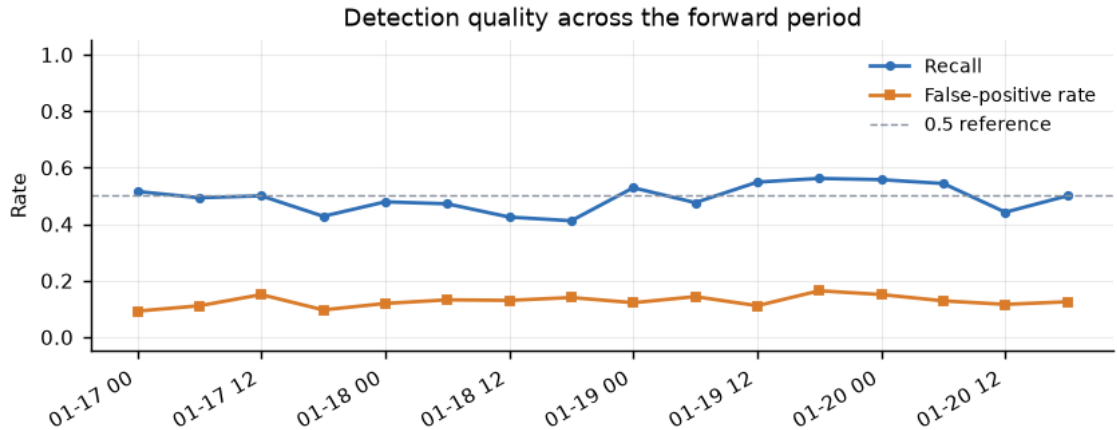
The findings below are computed from the prediction-level tables and drift events of this run. A model that produced no such finding would show an empty section rather than a plausible sentence.

logistic_regression

- F1 degradation +0.0104, recall degradation +0.0073, false-positive increase +0.0048.
- False positives: 716 on the forward test (mean score 0.910 when the model was wrong).
 - False negatives: 1,220 (mean score 0.707, of which 378 were scored above 0.8).
 - First drift alert: 2024-01-17 01:50:10.328737822.
 - Drift preceded the degradation window: False.
 - Strongest drift: total bytes, forward bytes, backward bytes, mean packet size, bytes per packet.
 - Of 32 alerting windows, 16 coincided with degraded performance and 16 did not. Drift and model failure are related but not the same event.

majority

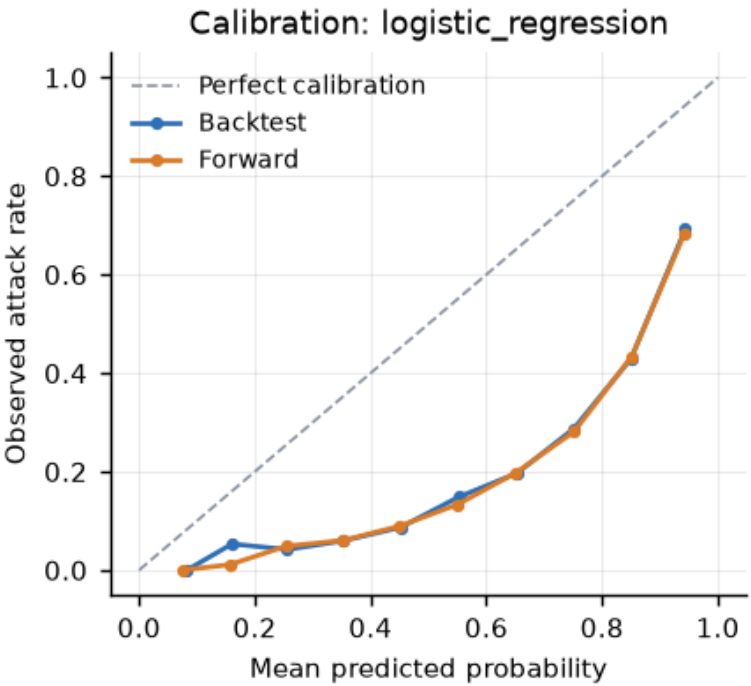
- F1 degradation +0.0028, recall degradation +0.0000, false-positive increase +0.0000.
- False positives: 5,603 on the forward test (mean score 1.000 when the model was wrong).
 - False negatives: 0 (mean score 0.000, of which 0 were scored above 0.8).
 - First drift alert: 2024-01-17 01:50:10.328737822.
 - Drift preceded the degradation window: False.
 - Strongest drift: total bytes, forward bytes, backward bytes, mean packet size, bytes per packet.
 - Of 32 alerting windows, 0 coincided with degraded performance and 32 did not. Drift and model failure are related but not the same



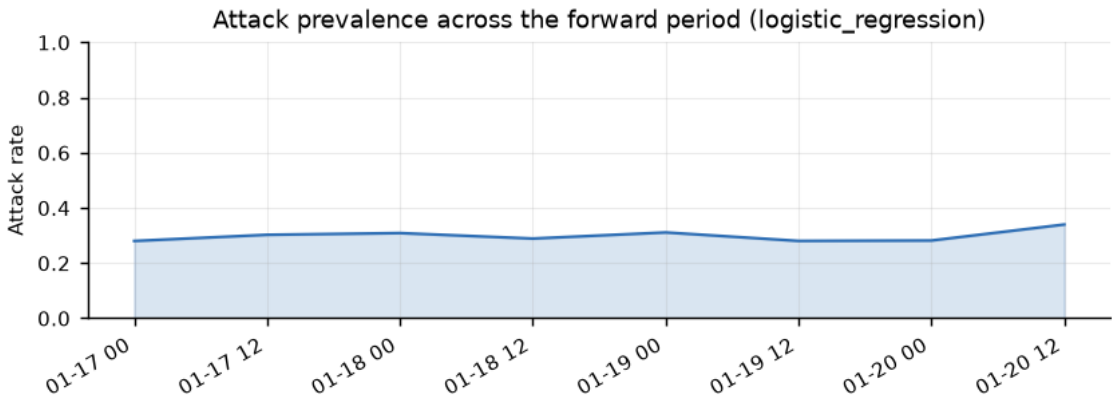
Recall and false-positive rate across the forward period.

Confidence is measured with the Brier score and expected calibration error on both periods, which shows whether a model that has become less accurate also becomes less honest about it.

Brier score
0.3738
logistic regression
0.0000



Reliability diagram, backtest versus forward period.



Attack prevalence across the forward period.

split mode

same rows, shuffled first; evaluates on contemporaneous traffic

feature policy

The common schema already excludes every payload-derived field (trans_depth, response_body_len, is_ftp_login, ct_ftp_cmd, ct_flw_http_mthd), so the metadata-only policy is already in force and there is no richer arm to compare against; the numbers reported here are the metadata-only detector

target fpr

adaptation window

unadapted forward performance

drift threshold

telemetry family

[illegible]

unsw_nb15

4 models on unsw_nb15, split 508,002 forward rows.

0.8200
0.9800 regression
0.8800 forest
0.8800 boosting

Comparing the two datasets directly is not valid: they have different label definitions, different attack taxonomies and different collection tools. The transferable claim is only about the method - splitting chronologically, fixing the operating point on validation data, and treating adaptation as a separate question from detection.

A detector trained on the earliest 20,000 flows of synthetic was evaluated on 8,000 flows it never saw. 2 of 2 models lost F1 on that later traffic. The largest loss was majority at +0.0028 F1; the best forward performer was logistic_regression at F1 0.5487.

The answer to the primary question is therefore conditional rather than binary. Models trained on historical traffic do continue to detect attacks on later traffic, but not at the level they reached on held-out data from the same period, and the drop is not uniform across models. The adaptation comparison further shows that refitting on recent data is not automatically an improvement, so the choice of strategy has to be made against a stated objective rather than assumed from its name.

- The numbers describe one dataset under one configuration. They are not a claim about modern networks, about adversarialness, or about deployment scenarios and the number was tested against a very specific adversary.
- These are one-pass results. Traffic from the same capture environment, so the shift it contains is whatever the environment itself had and a presumed stress test. The controlled shift experienced base business normal and extreme properties. Quality of the data and the quality of the results is not guaranteed and always involves a very small uncertainty on the shift as the attack class. The uncertainty numbers are point estimates without confidence intervals, so small differences between models should not be over-read.

```
Every run writes an immutable directory under results/experiments/. The one behind this report is
20260925T233012Z_temporal_synthetic_a55cdd.
experiment id : 20260925T233012Z_temporal_synthetic_a55cdd
dataset       : synthetic
git commit    : 78a18fc7ae23960339ba5683be6d042246a8dd7
feature schema : 98754df764d4fcf5
random seed   : 42
python        : 3.14.7

reproduce:
python -m pip install -e
python -m driftguard data.fetch --dataset unsw_nb15 --yes
python -m driftguard benchmark --config configs/benchmark.yaml
python -m driftguard report
```

Experiment directory contents

```
calibration_logistic_regression.json
calibration_majority.json
config.yaml
dataset_summary.json
drift_events.csv
failure_analysis_logistic_regression.json
failure_analysis_majority.json
figures.json
forward_windows_logistic_regression.csv
forward_windows_majority.csv
metadata.json
metrics.json
predictions_backtest_logistic_regression.csv
predictions_backtest_majority.csv
predictions_forward_logistic_regression.csv
predictions_forward_majority.csv
```

- A second dataset with a genuinely different capture environment, to separate temporal shift within a network from a controlled shift.
- Controlled shift experiments that attribute a degradation to one named property, such as packet size or protocol mix.
- A sequential drift detector that refits on a schedule and reports the cost of each refit, rather than retraining once at the end of calibration period.
- Calibration period alerting, so that a drop in expected calibration error can itself raise an operator alert even when the F1 has not yet moved.

Shyaa M., Ibrahim N., Zainol Z., Abdullah R., Anbar M., Alzubaidi L. (2024). Evolving cybersecurity frontiers: A comprehensive survey on concept drift and feature dynamics aware machine and deep learning in intrusion detection systems. *Engineering Applications of Artificial Intelligence*, 137, 109143. doi:10.1016/j.engappai.2024.109143

Macia-Fernandez G., Camacho J., Magan-Carrion R., Garcia-Teodoro P., Theron R. (2018). UGR'16: A new dataset for the evaluation of cyclostationarity-based network IDSSs. *Computers & Security*, 73, 411-424. doi:10.1016/j.cose.2017.11.004

Xu L., Han Z., Zhao D., Li X., Yu F., Chen C. (2024). Addressing concept drift in IoT anomaly detection: Drift detection, interpretation, and adaptation. *IEEE Transactions on Sustainable Computing*, 9(5), 913-924. doi:10.1109/tsusc.2024.3386667

Yang S., Zheng X., Li J., Xu J., Zhang X., Ngai E. (2025). Self-supervised adaptation method to concept drift for network intrusion detection. *IEEE Transactions on Dependable and Secure Computing*, 22(12), 7632-7646. doi:10.1109/tdsc.2025.3599321

UNSW Canberra School of Cyber Security. UNSW-NB15 network intrusion detection dataset. <https://research.unsw.edu.au/projects/unsw-nb15-dataset>

Network Security and Evolution Group (UGR). UGR'16 dataset. <https://nesg.ugr.es/nesg-ugr16/>